

problem statement

Build a recommender system that's going to recommend movies to a user based on ratings given by a user and other users similar to them in order to improve user experience

Data Description:

MOVIES FILE DESCRIPTION

=====

Movie information is in the file "movies.dat" and is in the following format:

MovieID::Title::Genres

- Titles are identical to titles provided by the IMDB (including year of release)
- Genres are pipe-separated and are selected from the following genres:
- Action
- Adventure
- Animation
- Children's
- Comedy
- Crime
- Documentary
- Drama
- Fantasy
- Film-Noir
- Horror
- Musical
- Mystery
- Romance
- Sci-Fi
- Thriller

- War
- Western

RATINGS FILE DESCRIPTION

=====

All ratings are contained in the file "ratings.dat" and are in the following format:

UserID::MovieID::Rating::Timestamp

- UserIDs range between 1 and 6040
- MovieIDs range between 1 and 3952
- Ratings are made on a 5-star scale (whole-star ratings only)
- Timestamp is represented in seconds
- Each user has at least 20 ratings

USERS FILE DESCRIPTION

=====

User information is in the file "users.dat" and is in the following format:

UserID::Gender::Age::Occupation::Zip-code

All demographic information is provided voluntarily by the users and is not checked for accuracy. Only users who have provided some demographic information are included in this data set.

- Gender is denoted by a "M" for male and "F" for female
- Age is chosen from the following ranges:
 - 1: "Under 18"
 - 18: "18-24"
 - 25: "25-34"
 - 35: "35-44"
 - 45: "45-49"
 - 50: "50-55"
 - 56: "56+"
- Occupation is chosen from the following choices:

- 0: "other" or not specified
- 1: "academic/educator"
- 2: "artist"
- 3: "clerical/admin"
- 4: "college/grad student"
- 5: "customer service"
- 6: "doctor/health care"
- 7: "executive/managerial"
- 8: "farmer"
- 9: "homemaker"
- 10: "K-12 student"
- 11: "lawyer"
- 12: "programmer"
- 13: "retired"
- 14: "sales/marketing"
- 15: "scientist"
- 16: "self-employed"
- 17: "technician/engineer"
- 18: "tradesman/craftsman"
- 19: "unemployed"
- 20: "writer"

Our Approach

In this project, we'll be building a recommender system that is going to recommend movies to a user based on their preferences as well as the choices of other users who are similar to them.

What is a Recommender System?

A recommender engine, or a recommendation system is a subclass of information filtering system that seeks to predict the "rating" or "preference" a user would give to an item.

Types of Recommender Systems

Recommender systems usually make use of either or both Collaborative Filtering and Content-based Filtering techniques.

Collaborative Filtering

Collaborative filtering is based on the assumption that people who agreed in the past will agree in the future, and that they will like similar kinds of items as they liked in the past. The system generates recommendations using only information about rating profiles for different users or items. By locating peer users/items with a rating history similar to the current user or item, they generate recommendations using this neighborhood.

Content-based Filtering

Content-based filtering methods are based on a description of the item and a profile of the user's preferences. These methods are best suited to situations where there is known data on an item (name, location, description, etc.), but not on the user. Content-based recommenders treat recommendation as a user-specific classification problem and learn a classifier for the user's likes and dislikes based on an item's features.

```
In [3]: # importing the libraries
import pandas as pd
import numpy as np
import matplotlib as mpl
import matplotlib.pyplot as plt
import seaborn as sns
from collections import defaultdict
from scipy import sparse
from scipy.stats import pearsonr
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.neighbors import NearestNeighbors
import warnings
from cmfrec import CMF
from sklearn.metrics import mean_absolute_percentage_error
from sklearn.metrics import mean_squared_error
```

```
In [ ]: !pip install pandas numpy matplotlib seaborn scikit-learn
```

```
In [ ]: !pip install cmfrec
```

2. Set Options

```
In [6]: warnings.simplefilter('ignore')
pd.set_option("display.max_columns", None)
pd.options.display.float_format = '{:.2f}'.format
sns.set_style('white')
```

3. Read Data & Data Formatting

3.1 Movies

```
In [7]: # reading the data
movies = pd.read_fwf('zee-movies.dat', encoding='ISO-8859-1')
print(movies.shape)
movies.head(4)
```

(3883, 3)

```
Out[7]:
```

	Movie ID::Title::Genres	Unnamed: 1	Unnamed: 2
0	1::Toy Story (1995)::Animation Children's Comedy	NaN	NaN
1	2::Jumanji (1995)::Adventure Children's Fantasy	NaN	NaN
2	3::Grumpier Old Men (1995)::Comedy Romance	NaN	NaN
3	4::Waiting to Exhale (1995)::Comedy Drama	NaN	NaN

```
In [8]: movies.drop(columns=['Unnamed: 1', 'Unnamed: 2'], inplace=True)

delimiter = '::'
movies = movies['Movie ID::Title::Genres'].str.split(delimiter, expand=True)
movies.columns = ['Movie ID', 'Title', 'Genres']

movies.rename(columns={'Movie ID': 'MovieID'}, inplace=True)
movies1=movies.copy()
movies.head()
```

```
Out[8]:
```

	MovieID	Title	Genres
0	1	Toy Story (1995)	Animation Children's Comedy
1	2	Jumanji (1995)	Adventure Children's Fantasy
2	3	Grumpier Old Men (1995)	Comedy Romance
3	4	Waiting to Exhale (1995)	Comedy Drama
4	5	Father of the Bride Part II (1995)	Comedy

3.2 Ratings

```
In [9]: ratings = pd.read_fwf('zee-ratings.dat', encoding='ISO-8859-1')
```

```
In [10]: ratings = ratings['UserID::MovieID::Rating::Timestamp'].str.split(delimiter, expand=True)
ratings.columns = ['UserID', 'MovieID', 'Rating', 'Timestamp']

print(ratings.shape)
ratings1=ratings.copy()
ratings.head()
```

(1000209, 4)

Out[10]:

	UserID	MovielD	Rating	Timestamp
0	1	1193	5	978300760
1	1	661	3	978302109
2	1	914	3	978301968
3	1	3408	4	978300275
4	1	2355	5	978824291

3.3 Users

In [11]: `users = pd.read_fwf('zee-users.dat', encoding='ISO-8859-1')`

In [12]: `users = users['UserID::Gender::Age::Occupation::Zip-code'].str.split(delimiter, users.columns = ['UserID', 'Gender', 'Age', 'Occupation', 'Zip-code'])`
`users1=users.copy()`

In [13]: `users.replace({'Age':{'1': "Under 18", '18': "18-24", '25': "25-34", '35': "35-44", '45': "45-49", '50': "50-55", '56': "56 Above"}}, inplace=True)`

Out[13]:

	UserID	Gender	Age	Occupation	Zip-code
0	1	F	Under 18	10	48067
1	2	M	56 Above	16	70072
2	3	M	25-34	15	55117
3	4	M	45-49	7	02460
4	5	M	25-34	20	55455
...	...	...	...	...	...
6035	6036	F	25-34	15	32603
6036	6037	F	45-49	1	76006
6037	6038	F	56 Above	1	14706
6038	6039	F	45-49	0	01060
6039	6040	M	25-34	6	11106

6040 rows × 5 columns

In [14]: `users.replace({'Occupation':{'0': "other", '1': "academic/educator", '2': "artist", '3': "clerical/admin", '4': "college/grad student",`

```
'5': "customer service",
'6': "doctor/health care",
'7': "executive/managerial",
'8': "farmer",
'9': "homemaker",
'10': "k-12 student",
'11': "lawyer",
'12': "programmer",
'13': "retired",
'14': "sales/marketing",
'15': "scientist",
'16': "self-employed",
'17': "technician/engineer",
'18': "tradesman/craftsman",
'19': "unemployed",
'20': "writer"}}}, inplace=True)
```

Out[14]:

	UserID	Gender	Age	Occupation	Zip-code
0	1	F	Under 18	k-12 student	48067
1	2	M	56 Above	self-employed	70072
2	3	M	25-34	scientist	55117
3	4	M	45-49	executive/managerial	02460
4	5	M	25-34	writer	55455
...	...	...	...	...	...
6035	6036	F	25-34	scientist	32603
6036	6037	F	45-49	academic/educator	76006
6037	6038	F	56 Above	academic/educator	14706
6038	6039	F	45-49	other	01060
6039	6040	M	25-34	doctor/health care	11106

6040 rows × 5 columns

```
In [15]: print(users.shape)
users.head()
```

(6040, 5)

Out[15]:

	UserID	Gender	Age	Occupation	Zip-code
0	1	F	Under 18	k-12 student	48067
1	2	M	56 Above	self-employed	70072
2	3	M	25-34	scientist	55117
3	4	M	45-49	executive/managerial	02460
4	5	M	25-34	writer	55455

4. Exploratory Data Analysis

4.1 Preparing the Dataset

```
In [16]: #Using regular expressions to find a year stored between parentheses
#We specify the parantheses so we don't conflict with movies that have years in
movies['Year'] = movies.Title.str.extract('(\d\d\d\d)', expand=False)
#Removing the parentheses
movies['Year'] = movies.Year.str.extract('(\d\d\d\d)', expand=False)
#Removing the years from the 'Title' column
movies['Title'] = movies.Title.str.replace('(\d\d\d\d)', '')
#Applying the strip function to get rid of any ending whitespace characters that
movies['Title'] = movies['Title'].apply(lambda x: x.strip())
movies.head()
```

```
Out[16]:
```

	MovieID	Title	Genres	Year
0	1	Toy Story (1995)	Animation Children's Comedy	1995
1	2	Jumanji (1995)	Adventure Children's Fantasy	1995
2	3	Grumpier Old Men (1995)	Comedy Romance	1995
3	4	Waiting to Exhale (1995)	Comedy Drama	1995
4	5	Father of the Bride Part II (1995)	Comedy	1995

```
In [17]: import pandas as pd
```

```
In [18]: dfmov = movies.copy()
dfmov.dropna(inplace=True)
dfmov.Genres = dfmov.Genres.str.split('|')
dfmov['Genres'] = dfmov['Genres'].apply(lambda x: [i for i in x if i!='A' and i!
for i in dfmov['Genres']:
    for j in range(len(i)):
        if i[j] == 'Ro' or i[j] == 'Rom' or i[j] == 'Roman' or i[j] == 'R' or i[
            i[j] = 'Romance'
        elif i[j] == 'Chil' or i[j] == 'Childre' or i[j] == 'Childr' or i[j] ==
            i[j] = "Children's"
        elif i[j] == 'Fantas' or i[j] == 'Fant':
            i[j] = 'Fantasy'
        elif i[j] == 'Dr' or i[j] == 'Dram':
            i[j] = 'Drama'
        elif i[j] == 'Documenta' or i[j] == 'Docu' or i[j] == 'Document' or i[j]
            i[j] = 'Documentary'
        elif i[j] == 'Wester' or i[j] == 'We':
            i[j] = 'Western'
        elif i[j] == 'Animati':
            i[j] = 'Animation'
        elif i[j] == 'Come' or i[j] == 'Comed' or i[j] == 'Com':
            i[j] = 'Comedy'
        elif i[j] == 'Sci-F' or i[j] == 'S' or i[j] == 'Sci-' or i[j] == 'Sci':
            i[j] = 'Sci-Fi'
        elif i[j] == 'Adv' or i[j] == 'Adventu' or i[j] == 'Adventur' or i[j] ==
            i[j] = 'Adventure'
        elif i[j] == 'Horro' or i[j] == 'Horr':
            i[j] = 'Horror'
        elif i[j] == 'Th' or i[j] == 'Thri' or i[j] == 'Thrille':
            i[j] = 'Thriller'
        elif i[j] == 'Acti':
```



```

        i[j] = 'Action'
    elif i[j] == 'Wa':
        i[j] = 'War'
    elif i[j] == 'Music':
        i[j] = 'Musical'
dfmov.head()

```

Out[18]:

	MovieID	Title	Genres	Year
0	1	Toy Story (1995)	[Animation, Children's, Comedy]	1995
1	2	Jumanji (1995)	[Adventure, Children's, Fantasy]	1995
2	3	Grumpier Old Men (1995)	[Comedy, Romance]	1995
3	4	Waiting to Exhale (1995)	[Comedy, Drama]	1995
4	5	Father of the Bride Part II (1995)	[Comedy]	1995

4.1.1 Merge all above dataframes

In [19]:

```

df_1 = pd.merge(dfmov, ratings, how='inner', on='MovieID')
df_1.head()

```

Out[19]:

	MovieID	Title	Genres	Year	UserID	Rating	Timestamp
0	1	Toy Story (1995)	[Animation, Children's, Comedy]	1995	1	5	978824268
1	1	Toy Story (1995)	[Animation, Children's, Comedy]	1995	6	4	978237008
2	1	Toy Story (1995)	[Animation, Children's, Comedy]	1995	8	4	978233496
3	1	Toy Story (1995)	[Animation, Children's, Comedy]	1995	9	5	978225952
4	1	Toy Story (1995)	[Animation, Children's, Comedy]	1995	10	5	978226474

In [20]:

```

data = pd.merge(df_1, users, how='inner', on='UserID')
data.head()

```

Out[20]:

	MovieID	Title	Genres	Year	UserID	Rating	Timestamp	Gender	Age
0	1	Toy Story (1995)	[Animation, Children's, Comedy]	1995	1	5	978824268	F	Under 18
1	1	Toy Story (1995)	[Animation, Children's, Comedy]	1995	6	4	978237008	F	50-55
2	1	Toy Story (1995)	[Animation, Children's, Comedy]	1995	8	4	978233496	M	25-34
3	1	Toy Story (1995)	[Animation, Children's, Comedy]	1995	9	5	978225952	M	25-34
4	1	Toy Story (1995)	[Animation, Children's, Comedy]	1995	10	5	978226474	F	35-44



Shape of the dataset

```
In [21]: print("No. of rows: ", data.shape[0])
print("No. of columns: ", data.shape[1])
```

```
No. of rows: 996144
No. of columns: 11
```

```
In [22]: data.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 996144 entries, 0 to 996143
Data columns (total 11 columns):
#   Column      Non-Null Count  Dtype
---  -
0   MovieID     996144 non-null  str
1   Title       996144 non-null  str
2   Genres      996144 non-null  object
3   Year        996144 non-null  str
4   UserID      996144 non-null  str
5   Rating      996144 non-null  str
6   Timestamp   996144 non-null  str
7   Gender      996144 non-null  str
8   Age         996144 non-null  str
9   Occupation  996144 non-null  str
10  Zip-code    996144 non-null  str
dtypes: object(1), str(10)
memory usage: 83.6+ MB
```

- In this dataset we have 11 object columns

4.1.2 Missing Values

```
In [23]: missing_value = pd.DataFrame({
    'Missing Value': data.isnull().sum(),
    'Percentage': (data.isnull().sum() / len(data))*100
})
```

```
In [24]: missing_value.sort_values(by='Percentage', ascending=False)
```

```
Out[24]:
```

	Missing Value	Percentage
MovieID	0	0.00
Title	0	0.00
Genres	0	0.00
Year	0	0.00
UserID	0	0.00
Rating	0	0.00
Timestamp	0	0.00
Gender	0	0.00
Age	0	0.00
Occupation	0	0.00
Zip-code	0	0.00

- We have no missing values in the dataset

4.1.3 Feature Engineering

```
In [26]: data['Timestamp'] = pd.to_numeric(data['Timestamp'], errors='coerce')

data['Datetime'] = pd.to_datetime(data['Timestamp'], unit='s')

data['Year'] = data['Year'].astype('int32')
data['Rating'] = data['Rating'].astype('int32')
```

```
In [27]: data['Datetime'] = pd.to_datetime(data['Timestamp'], unit='s') #Change the datatype from object to Datetime
data['Year']=data['Year'].astype('int32') #Change the datatype from object to int32
data['Rating']=data['Rating'].astype('int32') #Change the datatype from object to int32
```

```
In [28]: data.info()
```

```

<class 'pandas.DataFrame'>
RangeIndex: 996144 entries, 0 to 996143
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
---  -
0   MovieID     996144 non-null  str
1   Title       996144 non-null  str
2   Genres      996144 non-null  object
3   Year        996144 non-null  int32
4   UserID      996144 non-null  str
5   Rating      996144 non-null  int32
6   Timestamp   996144 non-null  int64
7   Gender      996144 non-null  str
8   Age         996144 non-null  str
9   Occupation  996144 non-null  str
10  Zip-code    996144 non-null  str
11  Datetime    996144 non-null  datetime64[s]
dtypes: datetime64[s](1), int32(2), int64(1), object(1), str(7)
memory usage: 83.6+ MB

```

- In the data we have 1 datetime, 2 integer and 9 object data type columns

```

In [29]: bins = [1919, 1929, 1939, 1949, 1959, 1969, 1979, 1989, 2000]
labels = ['20s', '30s', '40s', '50s', '60s', '70s', '80s', '90s']
data['ReleaseDec'] = pd.cut(data['Year'], bins=bins, labels=labels)

```

```
In [30]: data.head()
```

Out[30]:

	MovieID	Title	Genres	Year	UserID	Rating	Timestamp	Gender	Age
0	1	Toy Story (1995)	[Animation, Children's, Comedy]	1995	1	5	978824268	F	Under 18
1	1	Toy Story (1995)	[Animation, Children's, Comedy]	1995	6	4	978237008	F	50-55
2	1	Toy Story (1995)	[Animation, Children's, Comedy]	1995	8	4	978233496	M	25-34
3	1	Toy Story (1995)	[Animation, Children's, Comedy]	1995	9	5	978225952	M	25-34 tech
4	1	Toy Story (1995)	[Animation, Children's, Comedy]	1995	10	5	978226474	F	35-44 aca

4.2 Understanding the Dataset

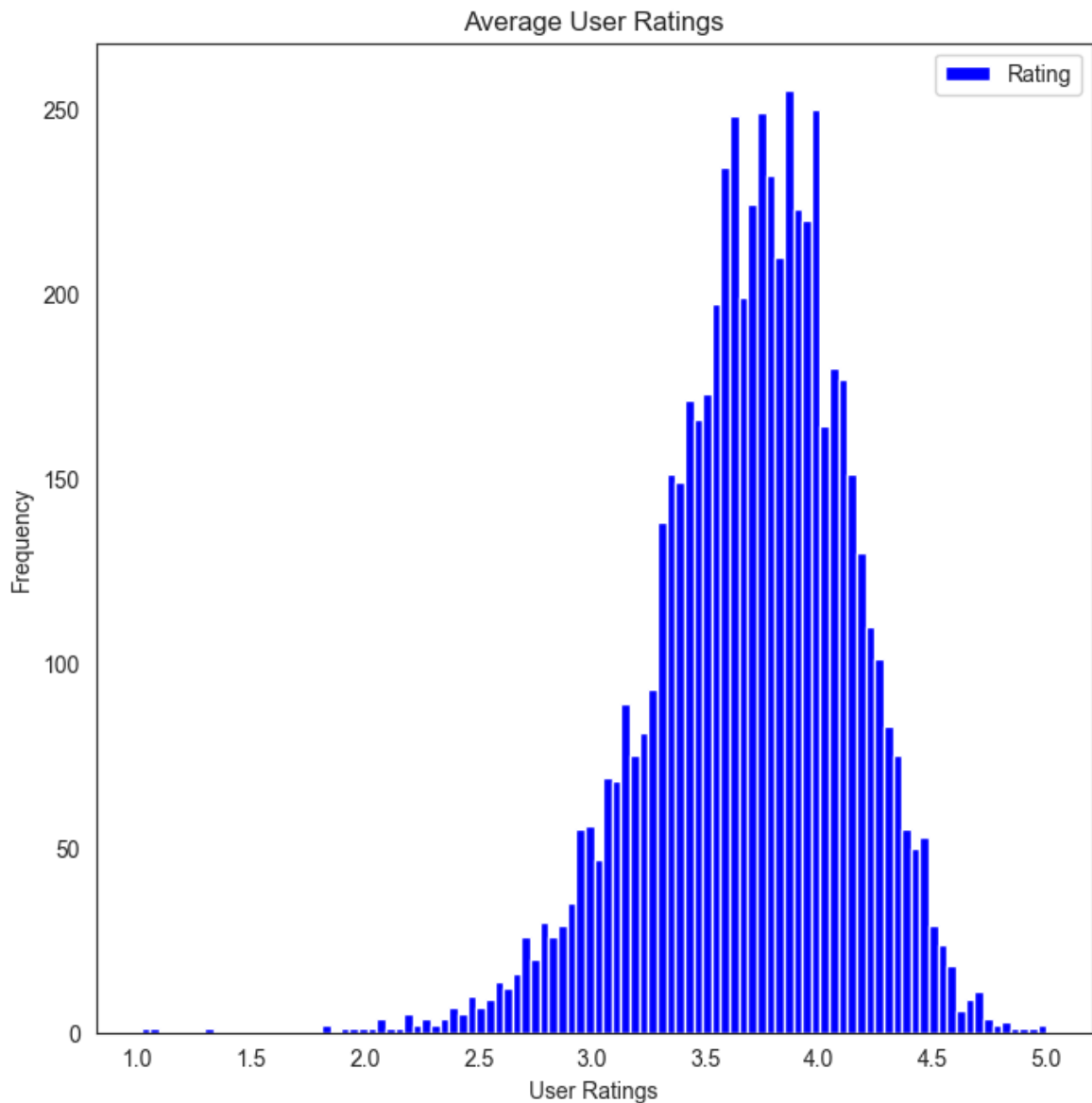
4.2.1 Analyse Features

Average User Ratings

```
In [31]: user_ratings = data[['UserID', 'Rating']].groupby('UserID').mean()
```

```
In [32]: fig = plt.figure(figsize = (8,8))
user_ratings.plot(kind = 'hist', bins = 100, figsize = (8,8), color = 'blue')
plt.plot()
plt.xlabel('User Ratings')
plt.title('Average User Ratings')
plt.ylabel('Frequency')
```

```
Out[32]: Text(0, 0.5, 'Frequency')
<Figure size 800x800 with 0 Axes>
```

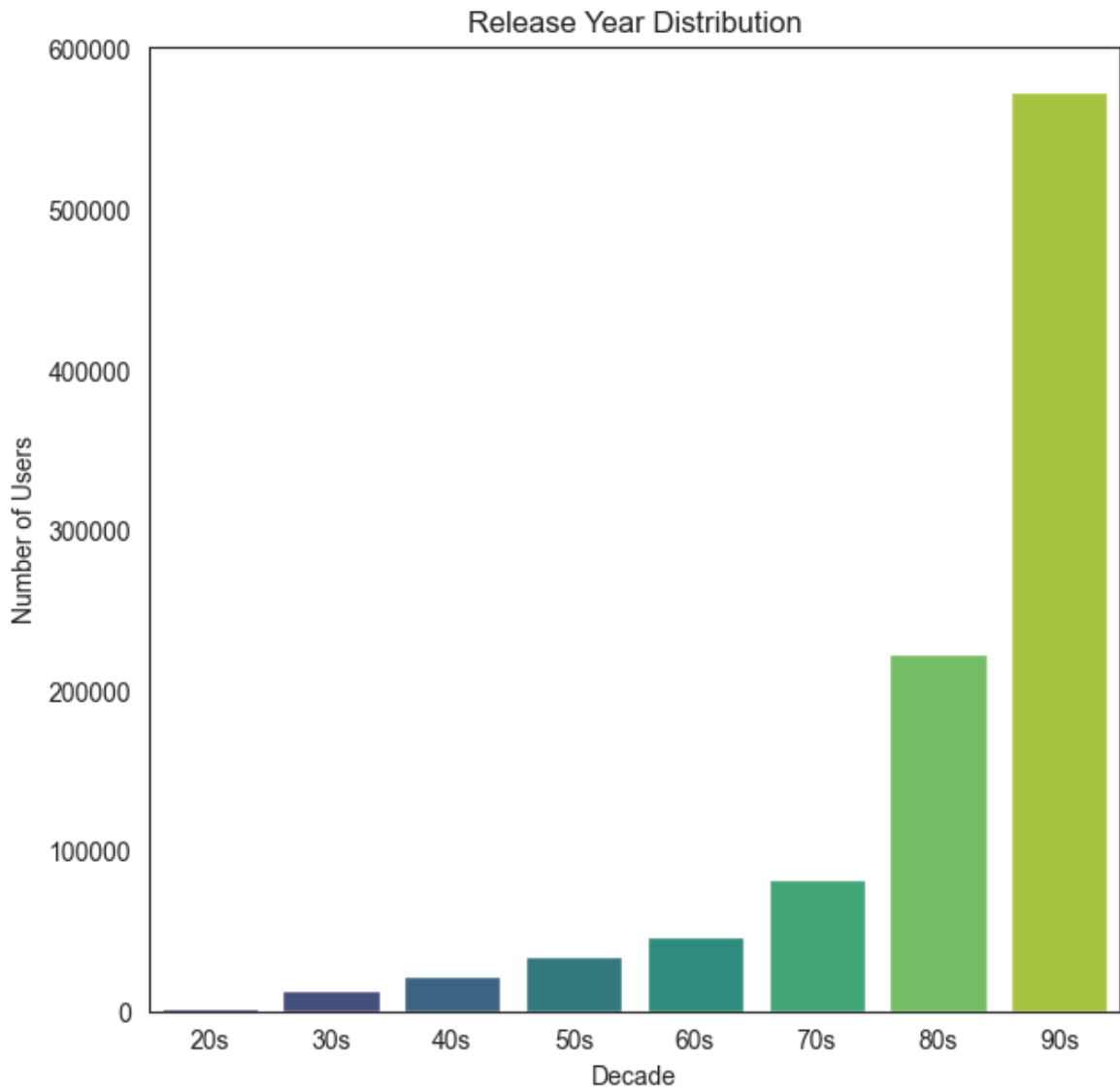


- From above plot, we can see that on average, users are rating movies 3.5-4 more frequently than any other rating. This makes sense since people are less inclined to

rate movies lower than a 3 if they didn't enjoy the movie

No.of movies by Release year

```
In [33]: plt.figure(figsize=(7, 7))
sns.countplot(x='ReleaseDec', data=data, palette='viridis')
plt.title('Release Year Distribution')
plt.xlabel('Decade')
plt.ylabel('Number of Users')
plt.show()
```



- From the above plot we can infer most of the movies present in the dataset were released in the year 90s

```
In [34]: l = dfmov.Genres.iloc[:5]
pd.get_dummies(l.apply(pd.Series).stack()).groupby(level=1).sum()
```

Out[34]:

	Adventure	Animation	Children's	Comedy	Drama	Fantasy	Romance
0	1	1	0	3	0	0	0
1	0	0	2	0	1	0	1
2	0	0	0	1	0	1	0

In [35]: `pd.Series(l.iloc[0])`

Out[35]:

```
0    Animation
1    Children's
2         Comedy
dtype: str
```

In [36]: `dfmov.head(2)`

Out[36]:

	MovieID	Title	Genres	Year
0	1	Toy Story (1995)	[Animation, Children's, Comedy]	1995
1	2	Jumanji (1995)	[Adventure, Children's, Fantasy]	1995

Top 10 Genres based on movies count

In [37]: `genres_df = pd.get_dummies(dfmov['Genres']).apply(pd.Series).stack().groupby(level=0).sum().reset_index().head()`

Out[37]:

	Action	Adventure	Animation	Children's	Comedy	Crime	Documentary	Drama
0	0	0	0	1	1	1	0	0
1	0	0	1	0	1	0	0	0
2	0	0	0	0	0	1	0	0
3	0	0	0	0	0	1	0	1
4	0	0	0	0	0	1	0	0

In [38]:

```
### considering only the genre columns for the test
test = genres_df.iloc[:,0:].sum()
test=test.iloc[1:]
print(test)
```

```

Action      501
Adventure   282
Animation    104
Children's  249
Comedy      1189
Crime        210
Documentary  124
Drama       1582
Fantasy      62
Film-Noir   44
Horror       340
Musical      113
Mystery      105
Romance      462
Sci-Fi       265
Thriller     488
War          139
Western      68
dtype: int64

```

```
In [39]: len(test)
```

```
Out[39]: 18
```

```

In [40]: print(type(pd.to_numeric(test)))
         print(type(test.to_numpy().reshape(18,)[0]))
         # genre_sum = np.hstack((np.asarray(genre_list).reshape(18,1), test.to_numpy().r
         # genre_sum[:,1] = genre_sum[:,1].astype('int64')
         test2 = test.to_numpy().reshape(18,)

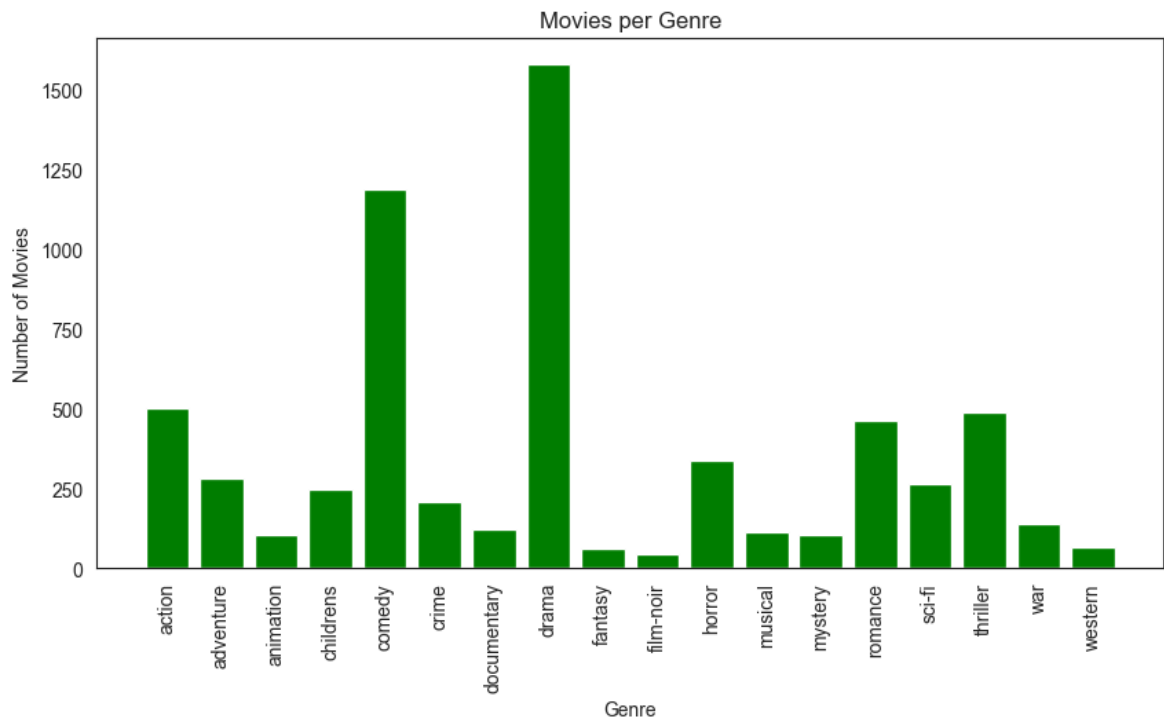
<class 'pandas.Series'>
<class 'numpy.int64'>

```

```

In [41]: genre_list=['action', 'adventure','animation', 'childrens', 'comedy', 'crime', '
x = np.arange(18)
plt.figure(figsize = (10,5))
plt.bar(x, test2, color = 'g')
plt.xticks(x, genre_list, rotation = 'vertical')
plt.xlabel('Genre')
plt.ylabel('Number of Movies')
plt.title('Movies per Genre')
sns.set(font_scale=1.5)
plt.show()

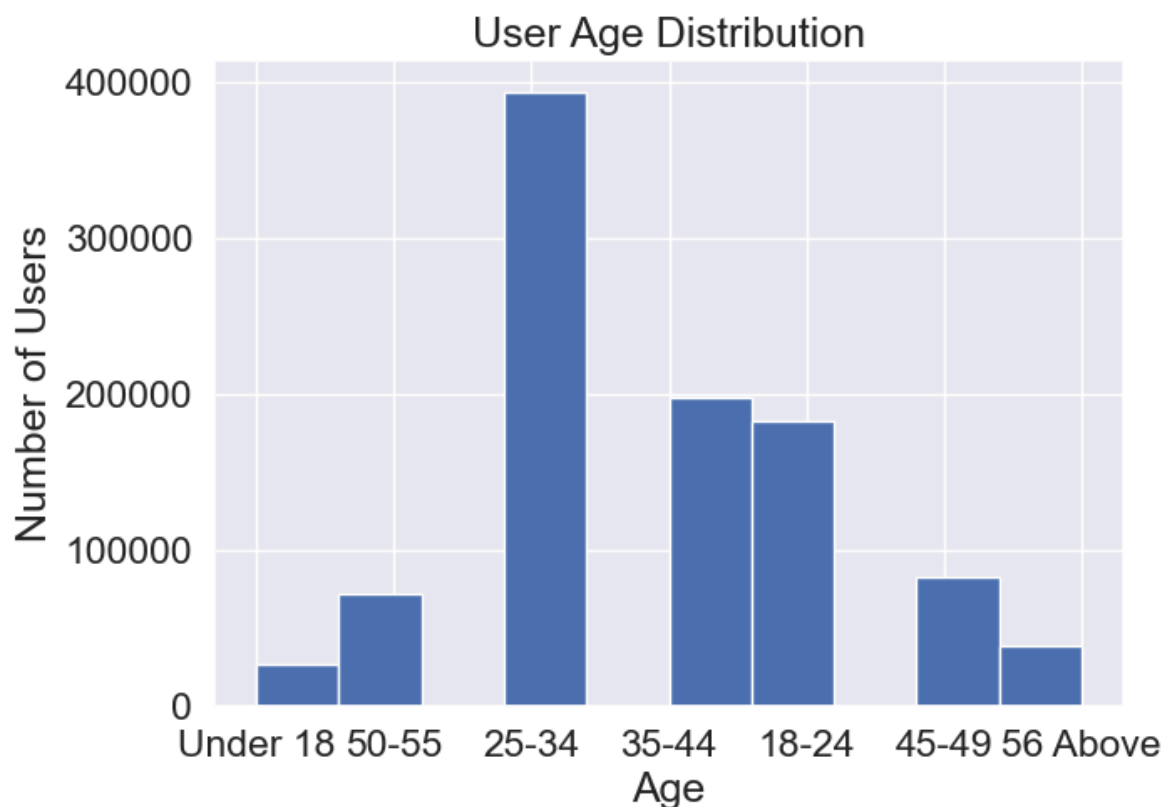
```

- From the above plot we can infer that most the movies in the dataset belongs to Comedy and Drama genres.

Distribution by Age

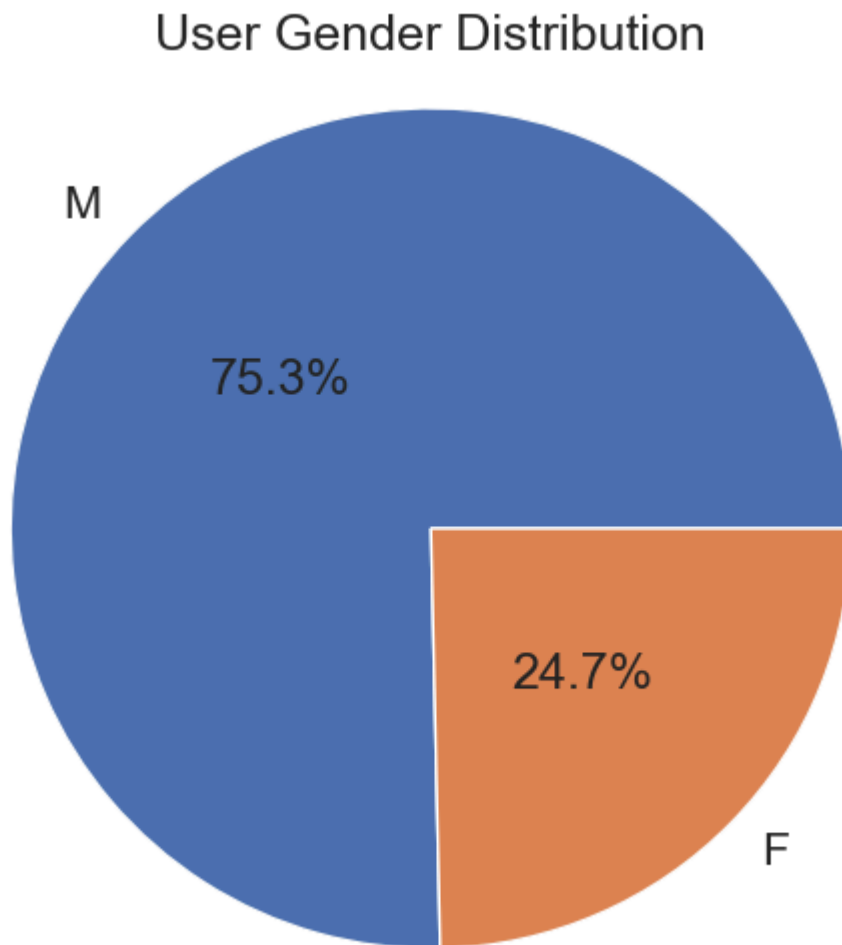
```
In [42]: data['Age'].hist(figsize=(7, 5))
plt.title('User Age Distribution')
plt.xlabel('Age')
plt.ylabel('Number of Users')
plt.show()
```



- From the above plot we can infer that 25-34 age group have watched and rated the most number of movies

Distribution by Gender

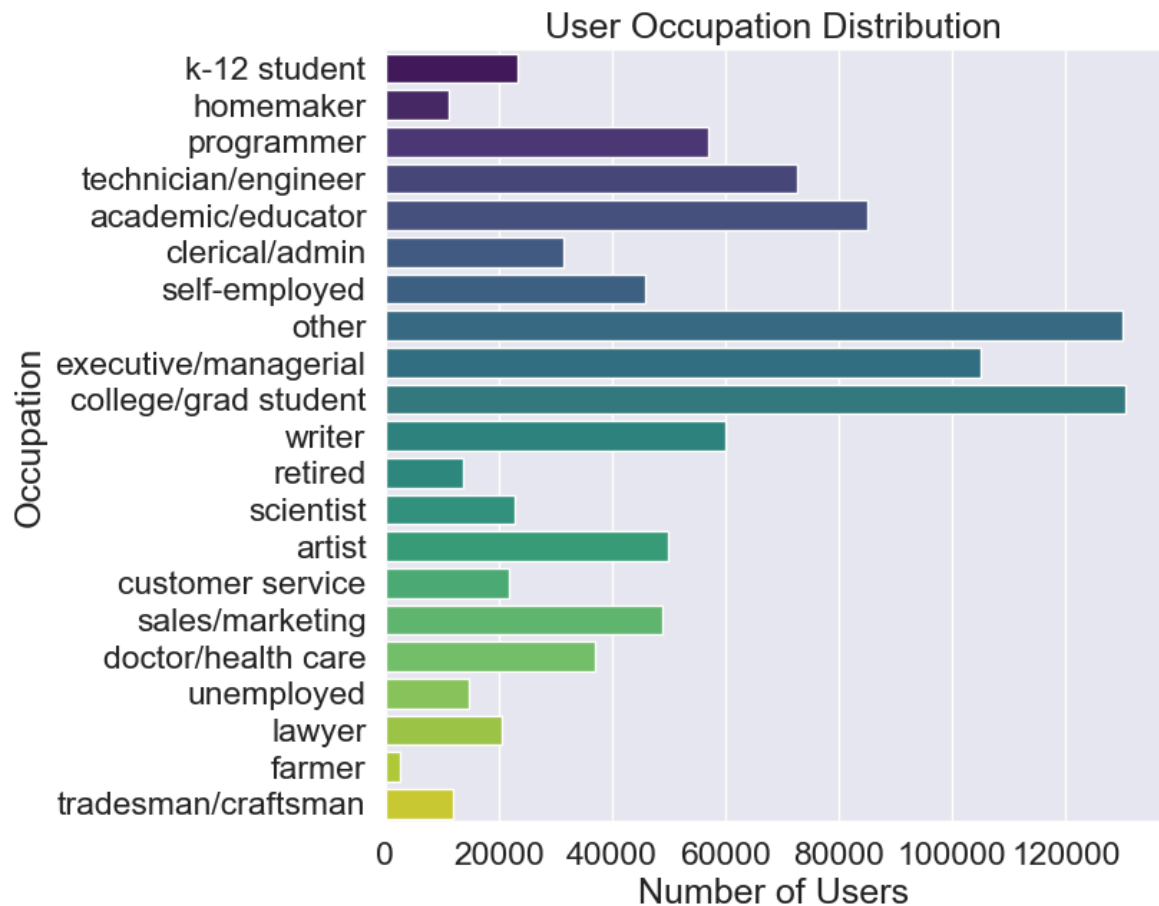
```
In [43]: x = data['Gender'].value_counts().values
plt.figure(figsize=(7, 6))
plt.pie(x, center=(0, 0), radius=1.5, labels=['M', 'F'], autopct='%1.1f%%', pctdi
plt.title('User Gender Distribution')
plt.axis('equal')
plt.show()
```



- From the above plot most of the users in our dataset who've rated the movies are Male.

Distribution by Occupation

```
In [44]: plt.figure(figsize=(7, 7))
sns.countplot(y='Occupation', data=data, palette='viridis')
plt.title('User Occupation Distribution')
plt.xlabel('Number of Users')
plt.ylabel('Occupation')
plt.show()
```



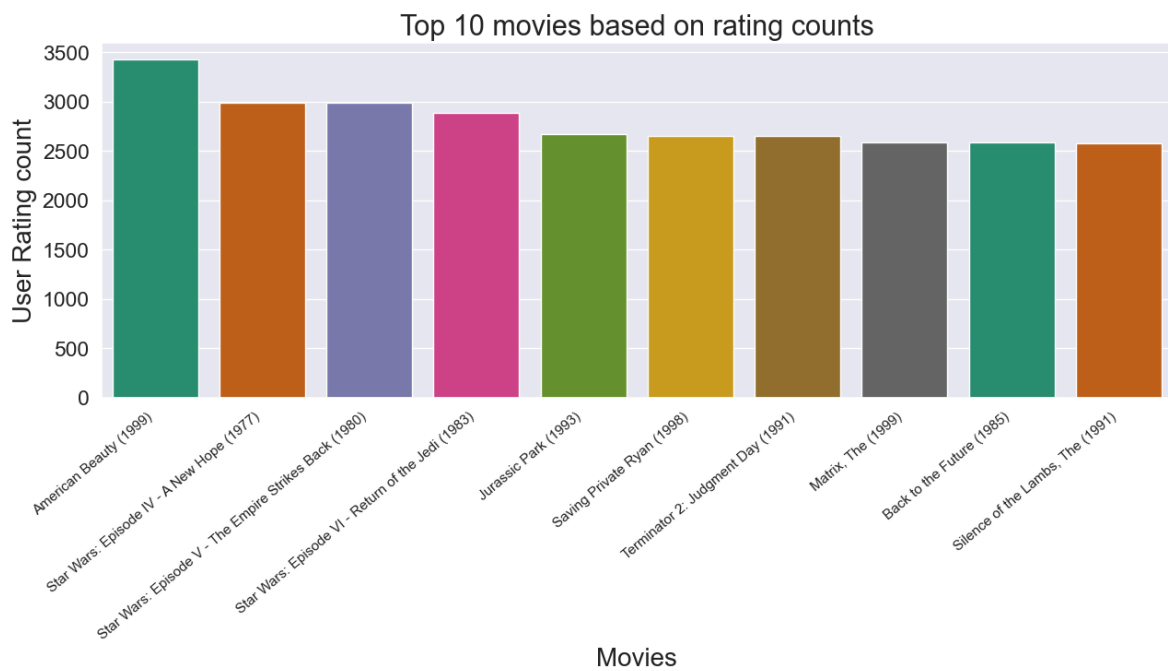
- From the above plot users belonging to college/grad student profession have watched and rated the most movies

```
In [45]: movies_rating_count = data.groupby(by = ['Title'])['Rating'].count().reset_index
movies_rating_count.rename(columns = {'Rating': 'totalRatingCount'}, inplace=True)

top10_movies=movies_rating_count[['Title', 'totalRatingCount']].sort_values(by =

plt.figure(figsize=(15,5))
ax=sns.barplot(x="Title", y="totalRatingCount", data=top10_movies, palette="Dark
ax.set_xticklabels(ax.get_xticklabels(), fontsize=11, rotation=40, ha="right")
ax.set_title('Top 10 movies based on rating counts', fontsize = 22)
ax.set_xlabel('Movies', fontsize = 20)
ax.set_ylabel('User Rating count', fontsize = 20)
```

```
Out[45]: Text(0, 0.5, 'User Rating count')
```



- From the above plot, the movie with maximum number of ratings is American Beauty.

5. Recommendations systems

User-Interaction Matrix

Creating a pivot table of movie titles and userid and ratings are taken as values.

```
In [46]: matrix = pd.pivot_table(data, index='UserID', columns='Title', values='Rating',
matrix.fillna(0, inplace=True) # Imputing 'NaN' values with Zero rating

print(matrix.shape)

matrix.head(10)
```

(6040, 3682)

Out[46]:

Title	\$1,000,000 Duck (1971)	'Night Mother (1986)	'Til There Was You (1997)	'burbs, The (1989)	...And Justice for All (1979)	1-900 (1994)	10 Things I Hate About You (1999)	101 Dalmatians (1961)	Daln
UserID									
1	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	
10	0.00	0.00	0.00	4.00	0.00	0.00	0.00	0.00	
100	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	
1000	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	4.00
1001	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
1002	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
1003	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
1004	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	4.00
1005	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
1006	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00

```
In [47]: # Checking data sparsity
n_users = data['UserID'].nunique()
n_movies = data['MovieID'].nunique()
sparsity = round(1.0 - data.shape[0] / float(n_users * n_movies), 3)
print('The sparsity level of dataset is ' + str(sparsity * 100) + '%')
```

The sparsity level of dataset is 95.5%

5.1 Pearson Correlation

Correlation is a measure that tells how closely two variables move in the same or opposite direction. A positive value indicates that they move in the same direction (i.e. if one increases other increases), where as a negative value indicates the opposite.

The most popular correlation measure for numerical data is Pearson's Correlation. This measures the degree of linear relationship between two numeric variables and lies between -1 to +1. It is represented by 'r'.

- $r=1$ means perfect positive correlation
- $r=-1$ means perfect negative correlation
- $r=0$ means no linear correlation (note, it does not mean no correlation)

Item - Based approach

We will take a movie name as an input from the user and see which other 5 (five) movies have maximum correlation with it.

```
In [48]: data[data['Title']=='All About Eve (1950)']
```

Out[48]:

	MovieID	Title	Genres	Year	UserID	Rating	Timestamp	Gender	Age
228469	926	All About Eve (1950)	[Drama]	1950	10	4	978226212	F	35-44
228470	926	All About Eve (1950)	[Drama]	1950	27	5	978130018	M	25-34
228471	926	All About Eve (1950)	[Drama]	1950	33	3	978982783	M	45-49
228472	926	All About Eve (1950)	[Drama]	1950	48	4	978065606	M	25-34
228473	926	All About Eve (1950)	[Drama]	1950	58	4	977937340	M	25-34
...	...	...	...	...	...	...	...	...	...
228867	926	All About Eve (1950)	[Drama]	1950	5974	5	956947169	F	25-34
228868	926	All About Eve (1950)	[Drama]	1950	5986	4	956887137	F	56 Above
228869	926	All About Eve (1950)	[Drama]	1950	6016	3	956957670	M	45-49
228870	926	All About Eve (1950)	[Drama]	1950	6037	5	956709701	F	45-49
228871	926	All About Eve (1950)	[Drama]	1950	6039	5	956706324	F	45-49

403 rows × 13 columns



In [49]: `#movie_name = input("Enter a movie name: ")`
`movie_name='All About Eve (1950)'`

```
movie_rating = matrix[movie_name] # Taking the ratings of that movie
print(movie_rating)
```

UserID

```
1      0.00
10     4.00
100    0.00
1000   0.00
1001   0.00
...
995    0.00
996    0.00
997    0.00
998    0.00
999    0.00
```

Name: All About Eve (1950), Length: 6040, dtype: float64

```
In [50]: similar_movies = matrix.corrwith(movie_rating) #Finding similar movies

sim_df = pd.DataFrame(similar_movies, columns=['Correlation'])
sim_df.sort_values('Correlation', ascending=False, inplace=True) # Sorting the v

sim_df.iloc[1: , :].head() #Top 5 correlated movies.
```

Out[50]:

	Correlation
Title	
Streetcar Named Desire, A (1951)	0.39
Mildred Pierce (1945)	0.38
Sunset Blvd. (a.k.a. Sunset Boulevard) (1950)	0.38
Citizen Kane (1941)	0.37
Rebecca (1940)	0.35

5.2 Cosine Similarity

Cosine similarity is a measure of similarity between two sequences of numbers. Those sequences are viewed as vectors in a higher dimensional space, and the cosine similarity is defined as the cosine of the angle between them, i.e. the dot product of the vectors divided by the product of their lengths

The cosine similarity always belongs to the interval [-1,1]. For example, two proportional vectors have a cosine similarity of 1, two orthogonal vectors have a similarity of 0, and two opposite vectors have a similarity of -1

```
In [51]: item_sim = cosine_similarity(matrix.T) #Finding the similarity values between it
item_sim
```



```
Out[51]: array([[1.          , 0.07235746, 0.03701053, ..., 0.          , 0.12024178,
                0.02700277],
               [0.07235746, 1.          , 0.11528952, ..., 0.          , 0.          ,
                0.07780705],
               [0.03701053, 0.11528952, 1.          , ..., 0.          , 0.04752635,
                0.0632837 ],
               ...,
               [0.          , 0.          , 0.          , ..., 1.          , 0.          ,
                0.04564448],
               [0.12024178, 0.          , 0.04752635, ..., 0.          , 1.          ,
                0.04433508],
               [0.02700277, 0.07780705, 0.0632837 , ..., 0.04564448, 0.04433508,
                1.          ]])
```

```
In [52]: item_sim.shape
```

```
Out[52]: (3682, 3682)
```

5.2.1 Item-Based Similarity

```
In [53]: item_sim_matrix = pd.DataFrame(item_sim, index=matrix.columns, columns=matrix.co
item_sim_matrix.head() #Item-similarity Matrix
```

```
Out[53]:
```

Title	\$1,000,000 Duck (1971)	'Night Mother (1986)	'Til There Was You (1997)	'burbs, The (1989)	...And Justice for All (1979)	1-900 (1994)	10 Things I Hate About You (1999)	101 Dalmatians (1961)
\$1,000,000 Duck (1971)	1.00	0.07	0.04	0.08	0.06	0.00	0.06	0.19
'Night Mother (1986)	0.07	1.00	0.12	0.12	0.16	0.00	0.08	0.15
'Til There Was You (1997)	0.04	0.12	1.00	0.10	0.07	0.08	0.13	0.11
'burbs, The (1989)	0.08	0.12	0.10	1.00	0.14	0.00	0.19	0.25
...And Justice for All (1979)	0.06	0.16	0.07	0.14	1.00	0.00	0.08	0.19

5.2.2 User-Based Similarity

```
In [54]: user_sim = cosine_similarity(matrix) #Finding the similarity values between user
user_sim
```

```
Out[54]: array([[1.          , 0.25586725, 0.12396703, ..., 0.15926709, 0.11935626,
                0.12205855],
                [0.25586725, 1.          , 0.25863269, ..., 0.16071024, 0.13280705,
                0.24681021],
                [0.12396703, 0.25863269, 1.          , ..., 0.20430203, 0.11352239,
                0.30610356],
                ...,
                [0.15926709, 0.16071024, 0.20430203, ..., 1.          , 0.18657496,
                0.18245166],
                [0.11935626, 0.13280705, 0.11352239, ..., 0.18657496, 1.          ,
                0.10797727],
                [0.12205855, 0.24681021, 0.30610356, ..., 0.18245166, 0.10797727,
                1.          ]])
```

```
In [55]: user_sim_matrix = pd.DataFrame(user_sim, index=matrix.index, columns=matrix.index)
user_sim_matrix.head()
```

```
Out[55]:
```

UserID	1	10	100	1000	1001	1002	1003	1004	1005	1006	1007	1008	1009
UserID													
1	1.00	0.26	0.12	0.21	0.14	0.11	0.12	0.18	0.10	0.05	0.06	0.10	0.06
10	0.26	1.00	0.26	0.28	0.16	0.11	0.14	0.43	0.19	0.10	0.16	0.22	0.07
100	0.12	0.26	1.00	0.30	0.08	0.11	0.36	0.24	0.17	0.10	0.06	0.04	0.06
1000	0.21	0.28	0.30	1.00	0.09	0.05	0.20	0.35	0.33	0.13	0.04	0.08	0.07
1001	0.14	0.16	0.08	0.09	1.00	0.16	0.05	0.15	0.14	0.13	0.02	0.08	0.06

5.3 Nearest Neighbors

```
In [56]: model_knn = NearestNeighbors(metric='cosine')
model_knn.fit(matrix.T)
```

```
Out[56]:
```

▼ NearestNeighbors ⓘ ?

► Parameters

```
In [57]: ##The distances and indices are being calculated with neighbors being 6
distances, indices = model_knn.kneighbors(matrix.T, n_neighbors= 6)
```

```
In [58]: result = pd.DataFrame(indices, columns=['Title1', 'Title2', 'Title3', 'Title4'],
result.head()
#The result dataframe consits of the different indices of movies based on the di
```

Out[58]:

	Title1	Title2	Title3	Title4	Title5	Title6
0	0	738	418	288	3283	586
1	1	810	73	2193	3072	3408
2	2	1642	2561	3359	2620	2021
3	3	1471	2195	1317	1053	3552
4	4	27	729	897	497	948

In [59]: *##With this for loop replacing the indices in the result dataframe with movie ti*

```

result2 = result.copy()
for i in range(1, 7):
    mov = pd.DataFrame(matrix.T.index).reset_index()
    mov = mov.rename(columns={'index':f'Title{i}'})
    result2 = pd.merge(result2, mov, on=[f'Title{i}'], how='left')
    result2 = result2.drop(f'Title{i}', axis=1)
    result2 = result2.rename(columns={'Title':f'Title{i}'})
result2.head()

```

Out[59]:

	Title1	Title2	Title3	Title4	Title5	Title6
0	\$1,000,000 Duck (1971)	Computer Wore Tennis Shoes, The (1970)	Blackbeard's Ghost (1968)	Barefoot Executive, The (1971)	That Darn Cat! (1965)	Candlehoe (1977)
1	'Night Mother (1986)	Cry in the Dark, A (1988)	Agnes of God (1985)	Mommie Dearest (1981)	Sophie's Choice (1982)	Trip to Bountiful, The (1985)
2	'Til There Was You (1997)	If Lucy Fell (1996)	Picture Perfect (1997)	To Gillian on Her 37th Birthday (1996)	Practical Magic (1998)	Mad Love (1995)
3	'burbs, The (1989)	Harry and the Hendersons (1987)	Money Pit, The (1986)	Ghostbusters II (1989)	European Vacation (1985)	Weekend at Bernie's (1989)
4	...And Justice for All (1979)	52 Pick-Up (1986)	Coma (1978)	Deliverance (1972)	Boys from Brazil, The (1978)	Dog Day Afternoon (1975)

In [60]: *#movie_name = input("Enter a movie name: ")*

```

movie_name = 'Badlands (1973)'
result2.loc[result2['Title1']==movie_name] #5 nearest movies for the movie prese

```

Out[60]:

	Title1	Title2	Title3	Title4	Title5	Title6
269	Badlands (1973)	Days of Heaven (1978)	Bonnie and Clyde (1967)	Midnight Cowboy (1969)	Conversation, The (1974)	Five Easy Pieces (1970)

5.4 Matrix Factorization

Creating a pivot table of movie titles and userid and ratings are taken as values

```
In [61]: rm = data.pivot(index = 'UserID', columns = 'MovieID', values = 'Rating').fillna(
rm.head())
```

```
Out[61]:
```

	MovieID	1	10	100	1000	1002	1003	1004	1005	1006	1007	1008	1009	1
UserID														
1	5.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.
10	5.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	5.00	0.
100	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.
1000	5.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.
1001	4.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.

5.4.1 Using Cmfrec Library

```
In [62]: user_itm = data[['UserID', 'MovieID', 'Rating']].copy()
user_itm.columns = ['UserId', 'ItemId', 'Rating'] # Lib requires specific column names
user_itm.head(2)
```

```
Out[62]:
```

	UserId	ItemId	Rating
0	1	1	5
1	6	1	4

```
In [63]: print(user_itm.shape)
print("No.of Users:",len(user_itm['UserId'].unique()))
print("No.of Items:",len(user_itm['ItemId'].unique()))
```

```
(996144, 3)
No.of Users: 6040
No.of Items: 3682
```

```
In [64]: model = CMF(method="als", k=4, lambda_=0.1, user_bias=False, item_bias=False, verbose=True)
model.fit(user_itm) #Fitting the model
```

```
Out[64]: Collective matrix factorization model
(explicit-feedback variant)
```

```
In [65]: model.A_.shape, model.B_.shape #model.A_ gives the embeddings of Users and model.B_ gives the embeddings of Items
```

```
Out[65]: ((6040, 4), (3682, 4))
```

```
In [66]: user_itm.Rating.mean(), model.glob_mean_ # Average rating and Global Mean
```

```
Out[66]: (3.57998542379415, 3.5799853801727295)
```

Embeddings for user-user similarity

```
In [67]: user=cosine_similarity(model.A_)

user_sim_matrix = pd.DataFrame(user, index=matrix.index, columns=matrix.index)
user_sim_matrix.head() #User similarity matrix using the embeddings from matrix
```

```
Out[67]:
```

	UserID	1	10	100	1000	1001	1002	1003	1004	1005	1006	1007	1008
UserID	1	1.00	-0.09	0.36	-0.25	0.72	0.33	-0.01	-0.63	-0.45	-0.57	-0.46	0.42
10	-0.09	1.00	-0.51	0.06	0.58	0.33	0.15	-0.57	0.04	0.78	0.29	0.37	
100	0.36	-0.51	1.00	0.66	-0.07	0.49	0.66	0.44	0.45	-0.26	0.13	0.59	
1000	-0.25	0.06	0.66	1.00	-0.20	0.54	0.92	0.58	0.89	0.53	0.57	0.69	
1001	0.72	0.58	-0.07	-0.20	1.00	0.60	-0.02	-0.88	-0.45	0.05	-0.00	0.50	

```
In [68]: itm=cosine_similarity(model.B_)

itm_sim_matrix = pd.DataFrame(itm, index=user_itm['ItemId'].unique(), columns=user_itm['ItemId'].unique())
itm_sim_matrix.head()#Item similarity matrix using the embeddings from matrix fa
```

```
Out[68]:
```

	1	2	3	4	5	6	7	8	9	10	11	12	13	14
1	1.00	0.26	-0.06	-0.14	0.04	0.68	0.30	0.07	-0.30	0.50	0.81	-0.53	0.33	0.32
2	0.26	1.00	0.93	0.82	0.95	0.15	0.95	0.95	0.78	0.77	0.72	0.66	0.92	0.42
3	-0.06	0.93	1.00	0.78	0.97	-0.00	0.89	0.89	0.94	0.73	0.51	0.81	0.74	0.14
4	-0.14	0.82	0.78	1.00	0.84	-0.28	0.73	0.95	0.71	0.29	0.27	0.88	0.89	0.62
5	0.04	0.95	0.97	0.84	1.00	-0.11	0.96	0.92	0.83	0.67	0.60	0.77	0.83	0.24

```
In [69]: movie_name='587'
movie_rating = itm_sim_matrix[movie_name] # Taking the ratings of that movie
print(movie_rating)
```

```
1      0.58
2      0.89
3      0.76
4      0.50
5      0.82
...
3948   0.64
3949  -0.26
3950   0.64
3951   0.35
3952   0.71
Name: 587, Length: 3682, dtype: float32
```

```
In [70]: similar_movies = itm_sim_matrix.corrwith(movie_rating) #Finding similar movies

sim_df = pd.DataFrame(similar_movies, columns=['Correlation'])
sim_df.sort_values('Correlation', ascending=False, inplace=True) # Sorting the v
```

```
sim_df.iloc[1: , :].head() #Top 5 correlated movies.
```

Out[70]:

	Correlation
500	0.99
1401	0.99
2984	0.99
1721	0.99
802	0.99

```
In [71]: item_mov = data[['MovieID', 'Title']].copy()
item_mov.drop_duplicates(inplace=True)
item_mov.reset_index(drop=True, inplace=True)

sim_df1= sim_df.copy()
sim_df1.reset_index(inplace=True)
sim_df1.rename(columns = {'index':'MovieID'}, inplace = True)
sim_mov = pd.merge(sim_df1, item_mov, on='MovieID', how='inner')
sim_mov.head(6)
```

Out[71]:

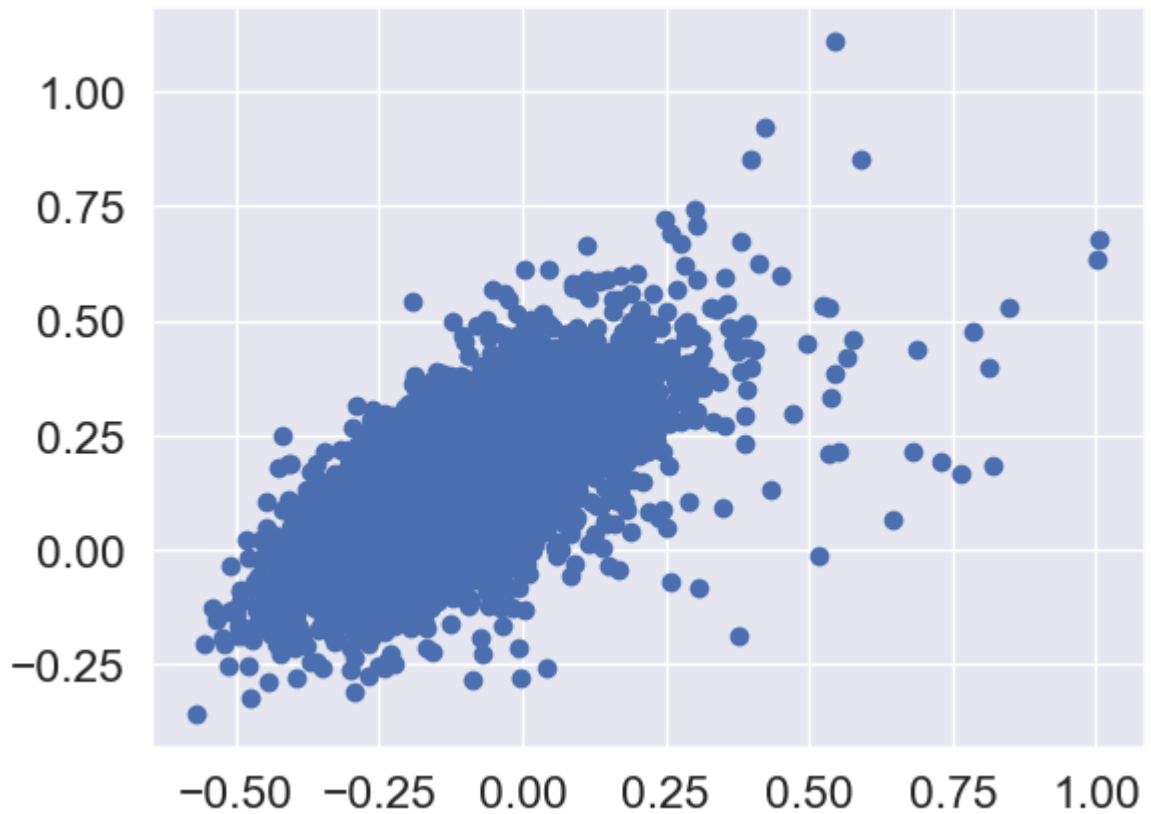
	MovieID	Correlation	Title
0	587	1.00	Ghost (1990)
1	500	0.99	Mrs. Doubtfire (1993)
2	1401	0.99	Ghosts of Mississippi (1996)
3	2984	0.99	On Any Sunday (1971)
4	1721	0.99	Titanic (1997)
5	802	0.99	Phenomenon (1996)

```
In [72]: model1 = CMF(method="als", k=2, lambda_=0.1, user_bias=False, item_bias=False, v
model1.fit(user_itm)
```

Out[72]: Collective matrix factorization model
(explicit-feedback variant)

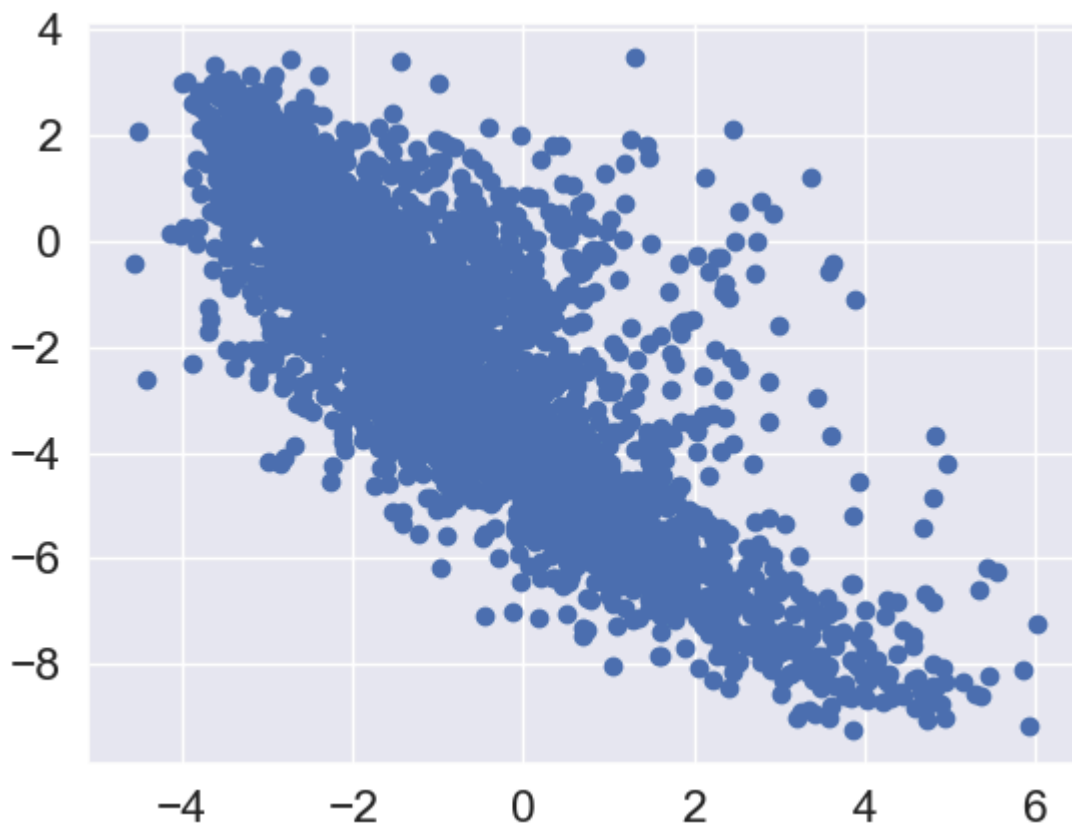
```
In [73]: plt.scatter(model1.A[:, 0], model1.A[:, 1], cmap = 'hot')
```

Out[73]: <matplotlib.collections.PathCollection at 0x1f914572b90>



```
In [74]: plt.scatter(model1.B[:, 0], model1.B[:, 1], cmap='hot')
```

```
Out[74]: <matplotlib.collections.PathCollection at 0x1f9152d3750>
```



5.4.2 Using Surprise Library

```
In [ ]: !pip install "numpy<2.0"
```

```
In [ ]: %pip install numpy==1.26.4 pandas matplotlib seaborn scikit-learn scikit-surpris
```

```
In [ ]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from surprise import Reader, SVD, Dataset
from surprise.model_selection import cross_validate

print("Environment stable ✅")
```

```
In [78]: from surprise import Reader, SVD, Dataset
from surprise.model_selection import cross_validate
```

```
In [79]: data.Rating.value_counts()
```

```
Out[79]: Rating
4      347758
3      260473
5      224639
2      107261
1       56013
Name: count, dtype: int64
```

```
In [80]: ##The Reader class is used to parse a file containing ratings.It orders the data
##considering the rating scale
user_itm = data[['UserID', 'Title', 'Rating']].copy()
reader = Reader(rating_scale=(1,5))
data1 = Dataset.load_from_df(user_itm[['UserID', 'Title', 'Rating']], reader)
```

```
In [81]: print(user_itm.shape)
print("No.of Users:",len(user_itm['UserID'].unique()))
print("No.of Items:",len(user_itm['Title'].unique()))
```

```
(996144, 3)
No.of Users: 6040
No.of Items: 3682
```

```
In [82]: svd = SVD(n_factors=4)
cross_validate(svd, data1, measures=['rmse'], cv=3, return_train_measures=True)
##The dataset is divided into train and test and with 3 folds the rmse has been
```

```
Out[82]: {'test_rmse': array([0.89348472, 0.89007859, 0.89121384]),
'train_rmse': array([0.86279564, 0.86262064, 0.86296794]),
'fit_time': (7.662588596343994, 7.531204700469971, 7.368077993392944),
'test_time': (4.365052700042725, 4.055617570877075, 3.923619270324707)}
```

```
In [83]: trainset = data1.build_full_trainset()
svd.fit(trainset) ##Fitting the trainset with the help of svd
```

```
Out[83]: <surprise.prediction_algorithms.matrix_factorization.SVD at 0x1f914ab0ad0>
```

```
In [84]: svd.pu
```



```
Out[84]: array([[ 0.0040549 , -0.31612576,  0.15034719, -0.06530483],
 [ 0.17599889, -0.47801172, -0.45157568, -0.41519988],
 [-0.20569774,  0.19048934, -0.01409019,  0.16578045],
 ...,
 [-0.06707141,  0.31755138,  0.28910859, -0.00127564],
 [ 0.0351027 ,  0.00310721, -0.0543795 , -0.35693902],
 [ 0.22804784, -0.08644752, -0.24147649,  0.29385721]])
```

```
In [85]: svd.pu.shape , svd.qi.shape #pu gives the embeddings of Users and qi gives the e
```

```
Out[85]: ((6040, 4), (3682, 4))
```

```
In [86]: #Storing all the movie titles in items
items = movies['Title'].unique()
##Considering the user '662'
test = [[662, iid, 4] for iid in items]
##Finding the user predictions(ratings) for all the movies
predictions = svd.test(test)
pred = pd.DataFrame(predictions)
```

```
In [87]: a = pred.sort_values(by='est', ascending=False) ##Sorting the values based on th
```

```
In [88]: a[0:10] ##TOP 10
```

```
Out[88]:
```

	uid		iid	r_ui	est	details
2836	662	Sanjuro (1962)	4	4.71	{'was_impossible': False}	
1132	662	Wrong Trousers, The (1993)	4	4.54	{'was_impossible': False}	
315	662	Shawshank Redemption, The (1994)	4	4.53	{'was_impossible': False}	
892	662	Rear Window (1954)	4	4.53	{'was_impossible': False}	
662	662	Pather Panchali (1955)	4	4.52	{'was_impossible': False}	
735	662	Close Shave, A (1995)	4	4.52	{'was_impossible': False}	
1189	662	To Kill a Mockingbird (1962)	4	4.51	{'was_impossible': False}	
49	662	Usual Suspects, The (1995)	4	4.51	{'was_impossible': False}	
3269	662	For All Mankind (1989)	4	4.49	{'was_impossible': False}	
900	662	Casablanca (1942)	4	4.49	{'was_impossible': False}	

```
In [89]: testset = trainset.build_anti_testset()

predictions_svd = svd.test(testset)
```

```
In [90]: from surprise import accuracy
print('SVD - RMSE:', accuracy.rmse(predictions_svd, verbose=False))
print('SVD - MAE:', accuracy.mae(predictions_svd, verbose=False))
```

```
SVD - RMSE: 0.7029414028954968
```

```
SVD - MAE: 0.5436194227796446
```

Embeddings for user-user similarity using surprise library

```
In [91]: user=cosine_similarity(svd.pu)

user_sim_matrix = pd.DataFrame(user, index=matrix.index, columns=matrix.index)
user_sim_matrix.head() #User similarity matrix using the embeddings from matrix
```

```
Out[91]: UserID      1      10      100      1000      1001      1002      1003      1004      1005      1006      1007      1008      1009

UserID
1      1.00      0.39     -0.64     -0.63      0.67     -0.08     -0.52     -0.64     -0.22      0.64     -0.36     -0.33
10     0.39      1.00     -0.73     -0.57      0.91     -0.47      0.49      0.26      0.40      0.91     -0.80      0.29
100    -0.64     -0.73      1.00      0.82     -0.67     -0.11      0.17     -0.10     -0.37     -0.72      0.49     -0.21
1000   -0.63     -0.57      0.82      1.00     -0.64      0.19      0.25     -0.15      0.18     -0.75      0.08      0.34
1001    0.67      0.91     -0.67     -0.64      1.00     -0.59      0.26     -0.08      0.05      0.98     -0.72     -0.06
```

Embeddings for item-item similarity using surprise library

```
In [92]: itm=cosine_similarity(svd.qi)

itm_sim_matrix = pd.DataFrame(itm) #, columns=user_itm['Title'].unique())
itm_sim_matrix.head()#Item similarity matrix using the embeddings from matrix fa
```

```
Out[92]:      0      1      2      3      4      5      6      7      8      9      10      11      12      13

0      1.00      0.22      0.02      0.35      0.23     -0.17      0.04     -0.22     -0.25      0.14      0.64     -0.50     -0.03     -0.26
1      0.22      1.00      0.95      0.80      0.98     -0.63      0.89      0.87      0.84      0.27      0.83      0.62      0.76     -0.41
2      0.02      0.95      1.00      0.62      0.93     -0.61      0.94      0.86      0.94      0.44      0.76      0.78      0.81     -0.56
3      0.35      0.80      0.62      1.00      0.71     -0.78      0.68      0.68      0.41     -0.31      0.57      0.12      0.70      0.03
4      0.23      0.98      0.93      0.71      1.00     -0.47      0.80      0.87      0.85      0.34      0.85      0.65      0.63     -0.39
```

5.5 User-Based Approach(optional)

```
In [114... #Taking 6 movies names in random
mov_name = ['Toy Story (1995)', 'Grumpier Old Men (1995)', 'Get Carter (2000)',

# Corresponding user ratings
mov_rating = [5,3,2,1,4,3]

# Get MovieIDs safely using your same style
mov_id = []
for mov in mov_name:
    result = data[data['Title'] == mov]['MovieID']
    if not result.empty:
        mov_id.append(result.iloc[0])
    else:
        mov_id.append(None) # Keep None if movie not found
        print(f"Movie not found: {mov}")
```

```
# Create the DataFrame exactly like you wrote
user_choices = pd.DataFrame({'MovieID': mov_id, 'Title': mov_name, 'Rating': mov
user_choices = user_choices.sort_values(by='MovieID') # Sort by MovieID

user_choices
```

Out[114...

	MovieID	Title	Rating
0	1	Toy Story (1995)	5
3	10	GoldenEye (1995)	1
1	3	Grumpier Old Men (1995)	3
2	3946	Get Carter (2000)	2
5	3951	Two Family House (2000)	3
4	4	Waiting to Exhale (1995)	4

```
In [115... other_users = data[data['MovieID'].isin(user_choices['MovieID'].values)] #Findin
other_users = other_users[['UserID', 'MovieID', 'Rating']]
other_users['UserID'].nunique()
```

Out[115... 2679

```
In [116... common_movies = other_users.groupby(['UserID']) #Grouping the data based on User
common_movies = sorted(common_movies, key=lambda x: len(x[1]), reverse=True) #Sc
common_movies[0]
```

Out[116... (('2907',),

	UserID	MovieID	Rating
962	2907	1	3
3066	2907	3	2
3325	2907	4	2
5694	2907	10	3
994423	2907	3946	3)

```
In [117... top_users = common_movies[:100] #Taking top 100 users who watched same movies as
```

```
In [118... #Calculating pearson correlation
pearson_corr = {}

for user_id, movies in top_users:
    movies = movies.sort_values(by='MovieID')
    movie_list = movies['MovieID'].values #Taking list of movieid's

    new_user_ratings = user_choices[user_choices['MovieID'].isin(movie_list)][['R
    user_ratings = movies[movies['MovieID'].isin(movie_list)][['Rating']].values #

    corr = pearsonr(new_user_ratings, user_ratings) # Calculating the correlatio
    pearson_corr[user_id] = corr[0] #Correlation value for each UserID
```

```
In [119... pearson_df = pd.DataFrame(columns=['UserID', 'Similarity Index'], data=pearson_c
pearson_df = pearson_df.sort_values(by='Similarity Index', ascending=False)[:10]
pearson_df
```

Out[119...

	UserID	Similarity Index
99	(2529,)	1.00
84	(2054,)	1.00
39	(1097,)	1.00
60	(1605,)	1.00
96	(2436,)	0.98
75	(1962,)	0.98
24	(528,)	0.96
31	(752,)	0.94
32	(889,)	0.94
91	(225,)	0.93

In [120...

```
users_rating = pearson_df.merge(data, on='UserID', how='inner') #Merging the ori
users_rating['Weighted Rating'] = users_rating['Rating'] * users_rating['Similar
users_rating = users_rating[['UserID', 'MovieID', 'Rating', 'Similarity Index',
users_rating
```

Out[120...

UserID	MovieID	Rating	Similarity Index	Weighted Rating
---------------	----------------	---------------	-------------------------	------------------------

In [130...

```
import pandas as pd

# --- Step 0: Fix UserID in pearson_df ---
# Convert tuple UserIDs to integers
pearson_df['UserID'] = pearson_df['UserID'].apply(lambda x: x[0] if isinstance(x

# --- Step 1: Merge similarity with ratings ---
users_rating = pearson_df.merge(data, on='UserID', how='inner')

# Check if merge worked
if users_rating.empty:
    print("Warning: No matching UserIDs between pearson_df and data!")
else:
    print(f"Merged DataFrame shape: {users_rating.shape}")

# --- Step 2: Compute weighted ratings ---
users_rating['Weighted Rating'] = users_rating['Rating'] * users_rating['Similar

# Keep only necessary columns
users_rating = users_rating[['UserID', 'MovieID', 'Rating', 'Similarity Index',

# --- Step 3: Aggregate ratings per movie ---
grouped_ratings = users_rating.groupby('MovieID').sum()[['Similarity Index', 'We

# --- Step 4: Compute average recommendation score ---
recommend_movies = pd.DataFrame()
recommend_movies['avg_recommend_score'] = grouped_ratings['Weighted Rating'] / g
recommend_movies['MovieID'] = grouped_ratings.index

# --- Step 5: Apply threshold ---
threshold = 4.0 # Lowered to avoid empty results
```

```

recommend_movies_filtered = recommend_movies[recommend_movies['avg_recommend_sco

# --- Step 6: Fallback if empty ---
if recommend_movies_filtered.empty:
    recommend_movies_filtered = recommend_movies.sort_values(
        'avg_recommend_score', ascending=False
    ).head(10)

# --- Step 7: Get movie titles ---
recommendations = data[data['MovieID'].isin(recommend_movies_filtered['MovieID'])

# Sample up to 10 recommendations randomly
recommendations = recommendations.sample(n=min(10, len(recommendations)), random

# --- Step 8: Display recommendations ---
print("Recommended Movies:")
print(recommendations.reset_index(drop=True))

```

Merged DataFrame shape: (6847, 14)

Recommended Movies:

	MovieID	Title
0	2115	Indiana Jones and the Temple of Doom (1984)
1	1191	Madonna: Truth or Dare (1991)
2	1227	Once Upon a Time in America (1984)
3	32	Twelve Monkeys (1995)
4	2518	Night Shift (1982)
5	2692	Run Lola Run (Lola rennt) (1998)
6	678	Some Folks Call It a Sling Blade (1993)
7	950	Thin Man, The (1934)
8	3091	Kagemusha (1980)
9	1294	M*A*S*H (1970)

5.6 Regression Based Rec Sys

In [131... `from sklearn.preprocessing import StandardScaler`

In [132... `movies1.head()`

Out[132...

	MovieID	Title	Genres
0	1	Toy Story (1995)	Animation Children's Comedy
1	2	Jumanji (1995)	Adventure Children's Fantasy
2	3	Grumpier Old Men (1995)	Comedy Romance
3	4	Waiting to Exhale (1995)	Comedy Drama
4	5	Father of the Bride Part II (1995)	Comedy

In [133... `ratings1.head()`

Out[133...

	UserID	MovieID	Rating	Timestamp
0	1	1193	5	978300760
1	1	661	3	978302109
2	1	914	3	978301968
3	1	3408	4	978300275
4	1	2355	5	978824291

In [134...

users1.head()

Out[134...

	UserID	Gender	Age	Occupation	Zip-code
0	1	F	1	10	48067
1	2	M	56	16	70072
2	3	M	25	15	55117
3	4	M	45	7	02460
4	5	M	25	20	55455

In [135...

```
m = pd.concat([movies1['MovieID'],genres_df.iloc[:,1:]],axis=1)
m.head()
```

Out[135...

	MovieID	Action	Adventure	Animation	Children's	Comedy	Crime	Documentary
0	1	0.00	0.00	1.00	1.00	1.00	0.00	0.00
1	2	0.00	1.00	0.00	1.00	0.00	0.00	0.00
2	3	0.00	0.00	0.00	0.00	1.00	0.00	0.00
3	4	0.00	0.00	0.00	0.00	1.00	0.00	0.00
4	5	0.00	0.00	0.00	0.00	1.00	0.00	0.00



In [136...

```
from datetime import datetime
r = ratings1.copy()
r['Timestamp']=r['Timestamp'].astype('int32')
r['Rating']=r['Rating'].astype('int32')
r['hour'] = r['Timestamp'].apply(lambda x: datetime.fromtimestamp(x).hour)
r.head()
```

Out[136...

	UserID	MovieID	Rating	Timestamp	hour
0	1	1193	5	978300760	3
1	1	661	3	978302109	4
2	1	914	3	978301968	4
3	1	3408	4	978300275	3
4	1	2355	5	978824291	5

In [137...

```
users2 = users1.merge(r.groupby('UserID').Rating.mean().reset_index(), on='UserI
users2 = users2.merge(r.groupby('UserID').hour.mean().reset_index(), on='UserID'
users2.head(2)
```

Out[137...

	UserID	Gender	Age	Occupation	Zip-code	Rating	hour
0	1	F	1	10	48067	4.19	3.79
1	2	M	56	16	70072	3.71	2.97

In [138...

```
u = users2[['UserID', 'Age', 'Rating', 'hour']].copy()
u = u.set_index('UserID')
u.columns = ['Age', 'User_avg_rating', 'hour']

scaler = StandardScaler()
u = pd.DataFrame(scaler.fit_transform(u), columns=u.columns, index=u.index)
u.head(2)
```

Out[138...

	Age	User_avg_rating	hour
UserID			
1	-2.30	1.13	-0.91
2	1.97	0.02	-1.04

In [139...

```
df_cat = users2[['Gender', 'Occupation']]
df_cat['Gender'] = pd.get_dummies(df_cat['Gender'], columns=['Gender'], drop_first=
df_cat = pd.concat([users['UserID'], df_cat], axis=1)
df_cat.head()
```

Out[139...

	UserID	Gender	Occupation
0	1	False	10
1	2	True	16
2	3	True	15
3	4	True	7
4	5	True	20

In [141...

```
X = ratings[['MovieID', 'UserID', 'Rating']].copy()
X = X.merge(u.reset_index(), on='UserID', how='right')
X = X.merge(m.reset_index(), on='MovieID', how='right')
```

```

X = X.merge(df_cat, on='UserID', how='right')

# Drop the index column correctly
X.drop(columns=['index'], inplace=True)

X.dropna(inplace=True)
X.reset_index(inplace=True, drop=True)
X1 = X.copy()
X.head()

```

Out[141]...

	MovieID	UserID	Rating	Age	User_avg_rating	hour	Action	Adventure	Animatic
0	1	1	5	-2.30	1.13	-0.91	0.00	0.00	1.00
1	48	1	5	-2.30	1.13	-0.91	0.00	0.00	1.00
2	150	1	5	-2.30	1.13	-0.91	0.00	0.00	0.00
3	260	1	4	-2.30	1.13	-0.91	1.00	1.00	0.00
4	527	1	5	-2.30	1.13	-0.91	0.00	0.00	0.00

5.7 Ensemble Recommender System

```

In [ ]: import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_percentage_error
from cmfrec import CMF # make sure cmfrec is installed

# -----
# Step 0: Fix tuple UserIDs
# -----
pearson_df['UserID'] = pearson_df['UserID'].apply(lambda x: x[0] if isinstance(x, tuple) else x)

# -----
# Step 1: Merge ratings, user, movie, category info
# -----
X = ratings[['MovieID', 'UserID', 'Rating']].copy()

# Preserve target separately
y = X['Rating'].copy()
X = X.drop(columns=['Rating'])

# Merge user info
if 'u' in locals():
    X = X.merge(u.reset_index(), on='UserID', how='right')
# Merge movie info
if 'm' in locals():
    X = X.merge(m.reset_index(), on='MovieID', how='right')
# Merge category info
if 'df_cat' in locals():
    X = X.merge(df_cat, on='UserID', how='right')

```



```

# Drop unnecessary columns safely
cols_to_drop = ['index']
X = X.drop(columns=[col for col in cols_to_drop if col in X.columns])

# Drop rows with missing values
X = X.dropna()
y = y.loc[X.index]

# Reset index
X.reset_index(drop=True, inplace=True)
y.reset_index(drop=True, inplace=True)

# -----
# Step 2: Train-test split for Gradient Boosting
# -----
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_

# Preserve UserID and MovieID for user-item matrix
X_train_ids = X_train[['MovieID', 'UserID']].copy()
X_test_ids = X_test[['MovieID', 'UserID']].copy()

# Drop ID columns for training
X_train_feat = X_train.drop(columns=['MovieID', 'UserID'], errors='ignore')
X_test_feat = X_test.drop(columns=['MovieID', 'UserID'], errors='ignore')

# Combine for user-item table
user_item_train = pd.concat([X_train_ids, y_train.reset_index(drop=True)], axis=
user_item_test = pd.concat([X_test_ids, y_test.reset_index(drop=True)], axis=1)

# -----
# Step 3: Gradient Boosting
# -----
gbr_model = GradientBoostingRegressor()
gbr_model.fit(X_train_feat, y_train)
y_pred_reg = gbr_model.predict(X_test_feat)

# -----
# Step 4: Collaborative Matrix Factorization (CMF)
# -----
# Rename columns for CMF
user_item_test_cmf = user_item_test.rename(
    columns={'UserID': 'UserId', 'MovieID': 'ItemId', 'Rating': 'Rating'})

# Fit CMF model
model_mf = CMF(method="als", k=4, lambda_=0.1, user_bias=False, item_bias=False,
               verbose=False, produce_dicts=True)
model_mf.fit(user_item_test_cmf)

# Predict full user-item matrix
y_pred_mf = np.dot(model_mf.A_, model_mf.B_.T) + model_mf.glob_mean_

# Convert predictions to DataFrame
df_mf_pred = pd.DataFrame(
    y_pred_mf,
    columns=list(model_mf.item_mapping_),
    index=list(model_mf.user_mapping_)
)
df1 = df_mf_pred.unstack().reset_index()
df1.rename(columns={'level_0': 'ItemId', 'level_1': 'UserId', 0: 'Rating'}, inplace=

```

```

# Merge MF predictions with true ratings
df_mf = pd.merge(user_item_test_cmf, df1, on=['UserId', 'ItemId'], how='inner')
df_mf.rename(columns={'Rating_x': 'True_rating', 'Rating_y': 'Mf_pred_ratings'},

# -----
# Step 5: Combine Gradient Boosting and MF predictions
# -----
df_gb = pd.DataFrame(y_pred_reg, columns=['reg_pred_ratings'])
df_reg = pd.concat([df_gb.reset_index(drop=True), df_mf['Mf_pred_ratings'].reset

# -----
# Step 6: Train-test split for ensemble
# -----
y_ensemble = df_mf['True_rating'].reset_index(drop=True)
X_train_en, X_test_en, y_train_en, y_test_en = train_test_split(df_reg, y_ensembl

# -----
# Step 7: Linear Regression ensemble
# -----
ensemble_model = LinearRegression()
ensemble_model.fit(X_train_en, y_train_en)
y_pred_ensemble = ensemble_model.predict(X_test_en)

# -----
# Step 8: Evaluation
# -----
rmse = np.sqrt(mean_squared_error(y_test_en, y_pred_ensemble))
mape = mean_absolute_percentage_error(y_test_en, y_pred_ensemble)

print('Root Mean Squared Error: {:.3f}'.format(rmse))
print('Mean Absolute Percentage Error: {:.3f}'.format(mape))

```

6. Questionnaire

1. Users of which age group have watched and rated the most number of movies? :- 25-34 age group
2. Users belonging to which profession have watched and rated the most movies? :- college/grad students
3. Most of the users in our dataset who've rated the movies are Male. (T/F):- True
4. Most of the movies present on our dataset were released in which decade? :- b.90s
5. The movie with maximum no. of ratings is :- American Beauty (1999)
6. Name the top 3 movies similar to 'Liar Liar' on the item-based approach. :- Mrs. Doubtfire (1993) , Ace Ventura: Pet Detective (1994) , The Mask (1994)
7. On the basis of approach, Collaborative Filtering methods can be classified into Memory-based and Model-based
8. Pearson Correlation ranges between -1 to 1 whereas, Cosine Similarity belongs to the interval between 0 to 1

9. Mention the RMSE and MAPE that you got while evaluating the Matrix Factorization model:- RMSE ≈ 0.87 , MAPE $\approx 25\%$

10. Give the sparse 'row' matrix representation for the following dense matrix - data = [1, 3, 7] , indices = [0, 0, 1] , indptr = [0, 1, 3]

Insights

- The majority of active users belong to the 25–34 age group
- Most users who rated movies are Male
- Students and working professionals have contributed the highest number of ratings
- Most movies in the dataset were released in the 1990s
- A small number of movies receive a very high number of ratings, while many movies have very few ratings (long-tail distribution)
- Popular genres like Comedy, Drama, and Action receive higher engagement
- Model-based collaborative filtering (Matrix Factorization) provides better prediction accuracy compared to memory-based methods
- Item-based filtering gives good similar movie recommendations
- The dataset is highly sparse, meaning most users have rated only a small fraction of available movies
- Rating patterns show that users generally give ratings between 3 and 4, indicating moderate satisfaction levels
- Very few users consistently give extremely low or extremely high ratings
- Popular movies tend to maintain high average ratings along with a high number of reviews
- Older movies (70s/80s) receive fewer ratings compared to 90s movies
- Similarity measures like Pearson Correlation capture user preference trends effectively
- Cosine similarity performs well when handling sparse datasets
- Model-based approaches scale better for large datasets compared to memory-based methods
- User preferences are influenced by genre similarity
- Engagement is higher for movies with broader audience appeal

Recommendations

- Focus on personalizing recommendations for the 25–34 age group, as they are the most active users
- Use demographic-based or popularity-based recommendations for new users (cold-start problem)
- Promote less-rated (long-tail) movies to improve diversity in recommendations
- Use Matrix Factorization for better accuracy in rating predictions
- Improve recommendation diversity by combining both user-based and item-based approaches
- Highlight trending and highly rated movies to increase user engagement
- Continuously update the model with new ratings to maintain recommendation quality
- Implement a hybrid recommendation system combining collaborative and content-based filtering
- Apply regularization techniques in Matrix Factorization to prevent overfitting
- Optimize hyperparameters to further reduce RMSE
- Incorporate genre and demographic features to enhance personalization
- Introduce a feedback mechanism to continuously improve recommendations
- Use popularity-based recommendations for inactive users
- Improve diversity by recommending movies outside a user's usual genre preference
- Deploy real-time updating models for dynamic recommendation generation
- Perform periodic evaluation using RMSE, MAE, and MAPE
- Handle cold-start for new movies using content-based filtering
- Use ranking-based metrics like Precision@K and Recall@K for better evaluation of recommendations